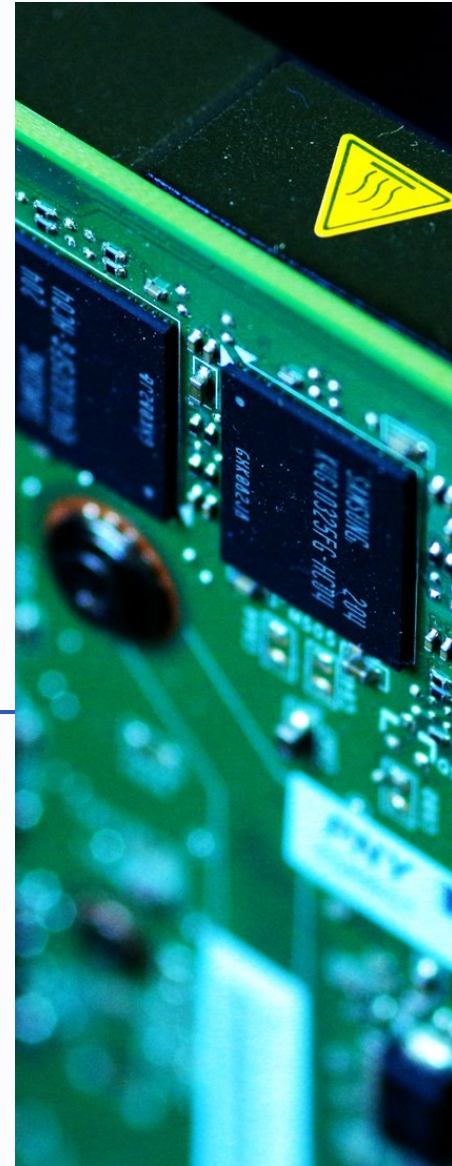


IREE: An end-to-end MLIR-Based Compiler and Runtime

LECTURE 6 · HARDWARE-SOFTWARE CO-DESIGN SCHOOL



IREE Makes Compiler Decisions Part of Co-Design

One cross-layer problem

END-TO-END

A model alone does not define execution. The compiler must choose partitions, schedules, memory behavior and target code; the runtime must load and coordinate that result.

Lecture question

How does a high-level model become a correct, efficient program for a particular machine?

Hardware becomes a target contract

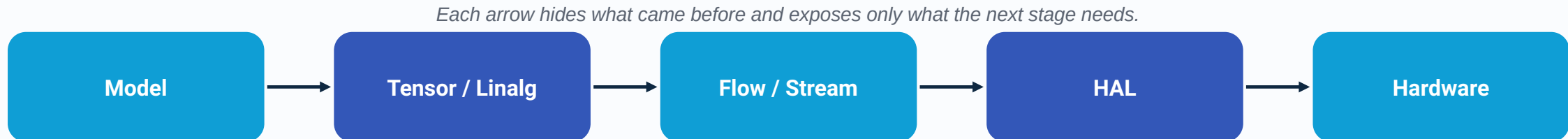
ISA features, vector width, memory hierarchy, device APIs and accelerator commands constrain legal and profitable lowering choices.

Software remains portable

The same imported MLIR can be specialized differently for CPUs, GPUs, RISC-V processors and custom accelerators.

Abstraction Hides Detail, Without Losing the Contract

Compilers work because each layer exposes what the next decision needs and hides everything else.



AI Deployment Is a Cross-Layer Optimization Problem

Performance emerges from choices made at several abstraction levels.

Model semantics

1

Operators, shapes, dtypes, control flow and data dependencies define legal transformations.

System schedule

2

Partitioning, dispatch formation, concurrency and synchronization determine when work runs.

Target code

3

Tiling, vectorization and instruction selection map work to a concrete ISA or device API.

Runtime behavior

4

Modules, buffers, drivers, commands and events connect compiled work to real hardware.

MLIR Is a Framework for Many Interoperable IRs

Multi-Level Intermediate Representation keeps useful structure alive long enough for the right optimization.

Core idea

Represent a program at several semantic levels

Extend the system through dialects

Transform operations with reusable passes and rewrite patterns

Lower progressively instead of jumping directly to machine code

Operation



A typed unit of computation with operands, results, regions and attributes.

Dialect



A coherent vocabulary for one abstraction level or target domain.

Pass



A transformation or analysis that changes or validates the IR.

Dialects Preserve the Right Information at Each Level

INTENT · STRUCTURE · MACHINE

A dialect should expose exactly the properties needed for its transformations and verification rules.



Lowering Is a Sequence of Legal, Verified Rewrites

HIGH LEVEL

MID LEVEL

LOW LEVEL

Preserve intent

Tensor operators, dynamic shapes and model-level dataflow remain visible.

Choose a schedule

Dispatches, tiles, loops, vectors, resources and synchronization become explicit.

Honor the target

Only operations, types, intrinsics and ABI rules supported by the target remain.

IREE Couples an MLIR Compiler with a Small Runtime

Compilation decides how work should execute; the runtime realizes that plan on the selected device.

IREE compiler

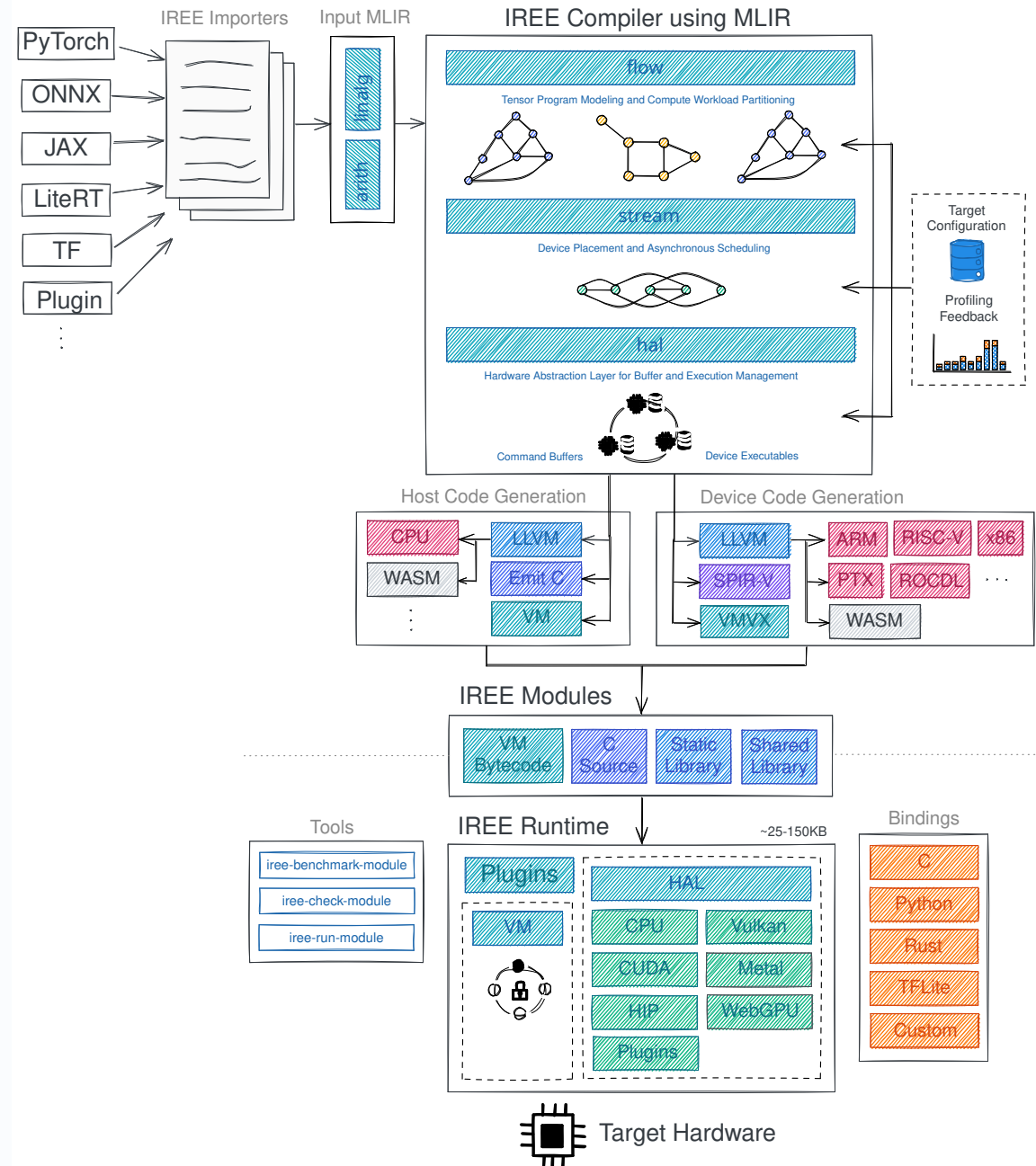
- Consumes imported MLIR
- Forms dispatches and optimizes tensor programs
- Schedules work and plans memory
- Generates device-specific executables
- Packages the result as a deployable module

→ IREE runtime

- Loads the compiled module and parameters
- Creates a HAL device
- Allocates or imports buffers
- Dispatches commands and synchronizes events
- Exposes C, Python and other application bindings

IREE Architecture Overview

IREE (Intermediate Representation Execution Environment¹) is an **MLIR**-based end-to-end compiler and runtime that lowers Machine Learning (ML) models to a unified IR that scales up to meet the needs of the datacenter and down to satisfy the constraints and special considerations of mobile and edge deployments.



One Imported Model Can Be Specialized for Many Targets

Common input

Imported MLIR separates the model from its original framework process.

Target specialization

Compiler flags select device, architecture, features, layouts and codegen strategy.

Portable deployment

VMFB plus the runtime carries host logic and target executables to the system.

IREE Compilation Pipeline (MLIR-Based)

1. High-Level Model



Examples:

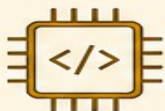
- TensorFlow
- PyTorch
- ONNX
- JAX

2. MLIR-Based Compiler Pipeline



Uses MLIR to represent and transform the model through multiple passes.

3. Device-Specific Code Generation



Generate optimized code tailored to the target device (CPU, GPU, Accelerator).

4. Portable Runtime Module



Produces a self-contained runtime module (IREE Module) that can be deployed.

MLIR is the core enabler for representation, optimization, and lowering.

IREE Runtime

5. Load & Execute at Runtime



The IREE runtime loads the module, manages resources, and executes efficiently.

6. Portable & Efficient Execution



Optimal performance across different hardware backends.

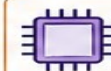
Where It Runs



CPUs



GPUs



Embedded Devices



Custom Accelerators

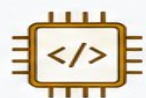
End-to-End Flow



Model



MLIR Pipeline



Device Code Generation



IREE Module (Portable)



IREE Runtime



Run on Any Supported Hardware

Key Takeaway

IREE takes a high-level MIL model → compiles it using MLIR → generates device-specific code → packages it into a portable runtime module → and executes it efficiently on a wide range of hardware targets.

AOT vs JIT Compilation

First, we need to understand the difference here!

Ahead of time (AOT)

- Export the model outside the Python process
- Compile a target-specific VMFB before deployment
- Bundle a small runtime with the application
- Best fit for reproducible edge, server and embedded delivery

Just in time (JIT) [browsers]

- Use IREE through torch.compile and Turbine
- Capture and compile within an interactive Python session
- Convenient for development and rapid experiments
- Deployment outside Python returns to the AOT workflow

IREE Accepts Several Frontend Routes

CAPTURE → IMPORT → LEGALIZE

Choose the route that preserves your framework semantics and produces the cleanest supported MLIR boundary.

PyTorch



IREE Turbine supports JIT through `torch.compile` and AOT export to MLIR.

ONNX

ONNX

`iree-import-onnx` converts an ONNX graph to imported MLIR before core compilation.

JAX



JAX exports StableHLO-oriented programs that can enter the IREE pipeline.

TensorFlow / LiteRT



TensorFlow and TensorFlow Lite models use their framework-specific export/import paths.

Read Imported MLIR as a Typed Dataflow Program

Start with the function signature, then trace values through tensor operations and return types.

Representative structure

```
func.func @main(%arg0: tensor<1x3x224x224xf32>)  
  -> tensor<1x1000xf32> {  
    %0 = ... : tensor<1x3x224x224xf32>  
    %1 = linalg... ins(%0, ...) outs(...)  
    return %1 : tensor<1x1000xf32>  
  }
```

What to inspect

Entry-point name and ABI-visible inputs

Static versus dynamic dimensions

Element types and quantization boundaries

Constants or external parameter references

Unexpected fallback or unsupported operations

Capture Freezes Python Behavior into a Graph

EAGER MODEL

CAPTURE

EXPORTED PROGRAM

Run Python

Modules, control flow and tensor operations execute through the framework.

Observe the program

Representative inputs expose tensor operations, shapes, parameters and data dependencies.

Create a stable boundary

The graph can now be imported and compiled independently of ordinary Python execution.

Part 2 · Lower Through the IREE Compiler

Move from tensor intent to dispatches, scheduled resources, device commands and hardware-specific code.



IREE Lowers a Model Through Four System Layers

The first half of the pipeline turns model semantics into executable, scheduled work.

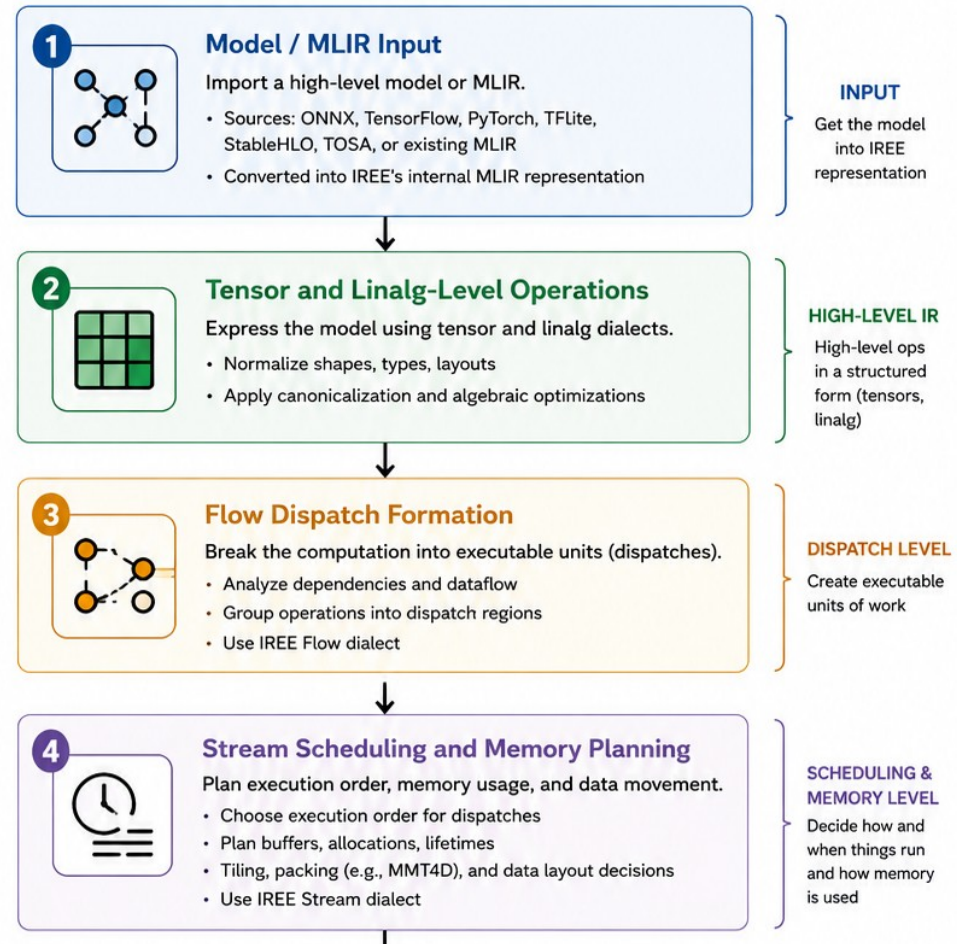
Progressive responsibilities

Import: model / MLIR input

Tensor + Linalg: global optimization

Flow: dispatch formation

Stream: scheduling and memory planning



Flow Turns Tensor Work into Dispatch Regions

Flow separates host dataflow from dense compute that can be compiled and launched as device work.

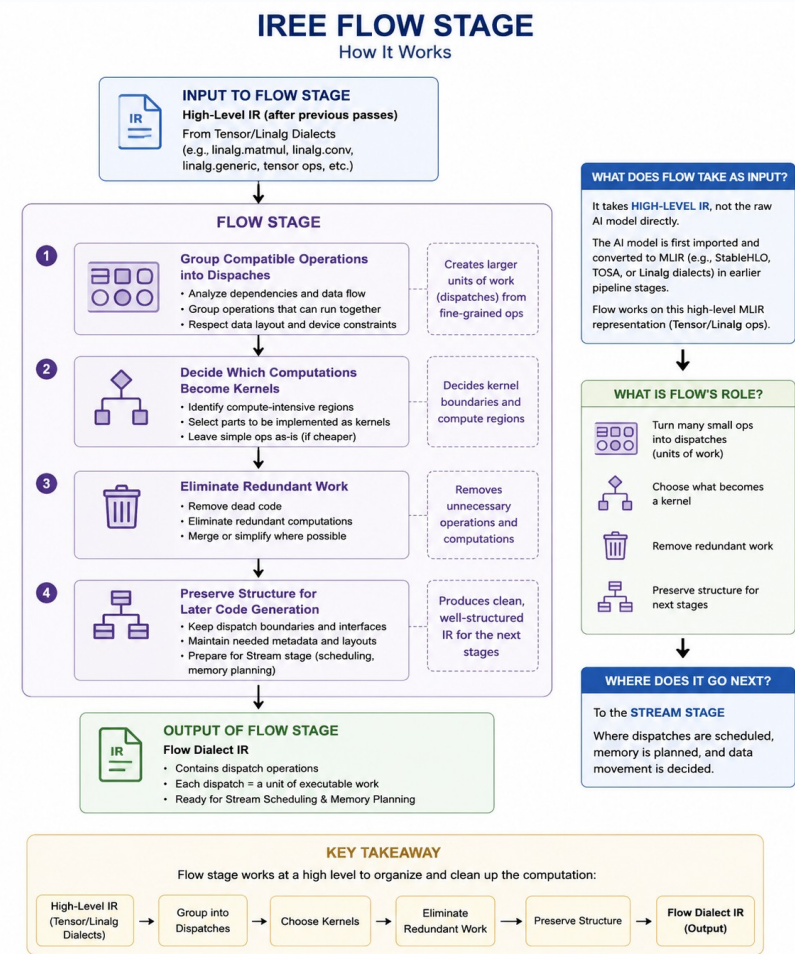
Flow decisions

Identify dispatchable tensor computation

Fuse producers and consumers where profitable

Materialize dispatch regions and workload shapes

Preserve dependencies for later scheduling



Stream Schedules Work and Plans Memory

Stream converts tensor programs into explicitly scheduled asynchronous work over opaque resources.

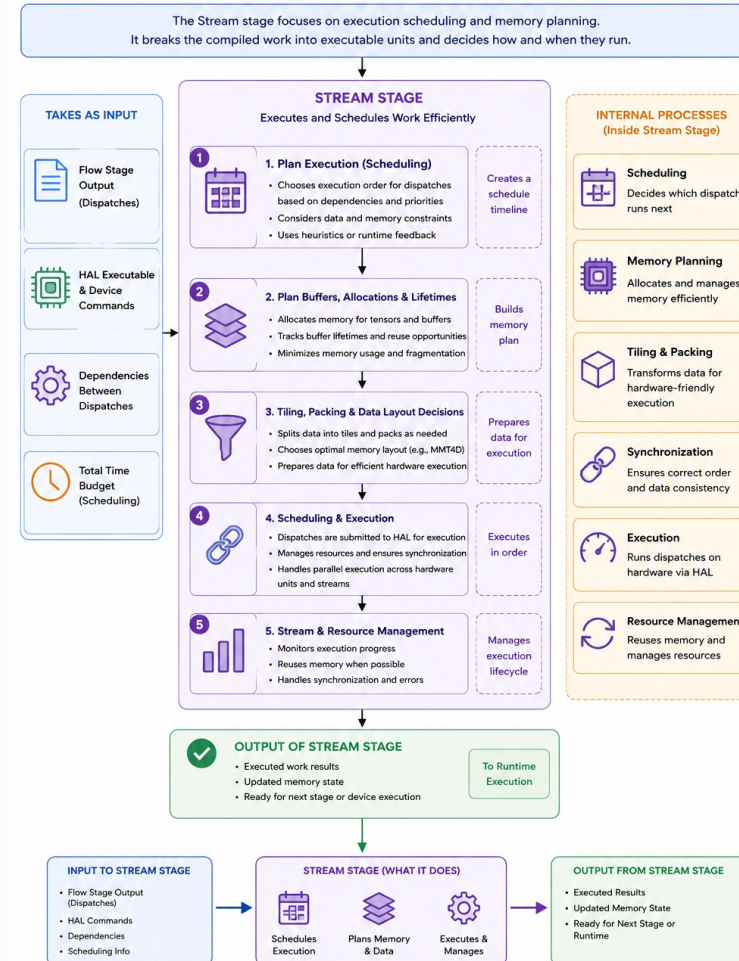
Stream decisions

Partition work across target affinities

Encode tensors into target-specific resources

Assign lifetimes and plan allocations

Schedule concurrent work and synchronization



HAL Makes Devices and Commands Explicit

The Hardware Abstraction Layer provides one compiler/runtime contract across heterogeneous devices.

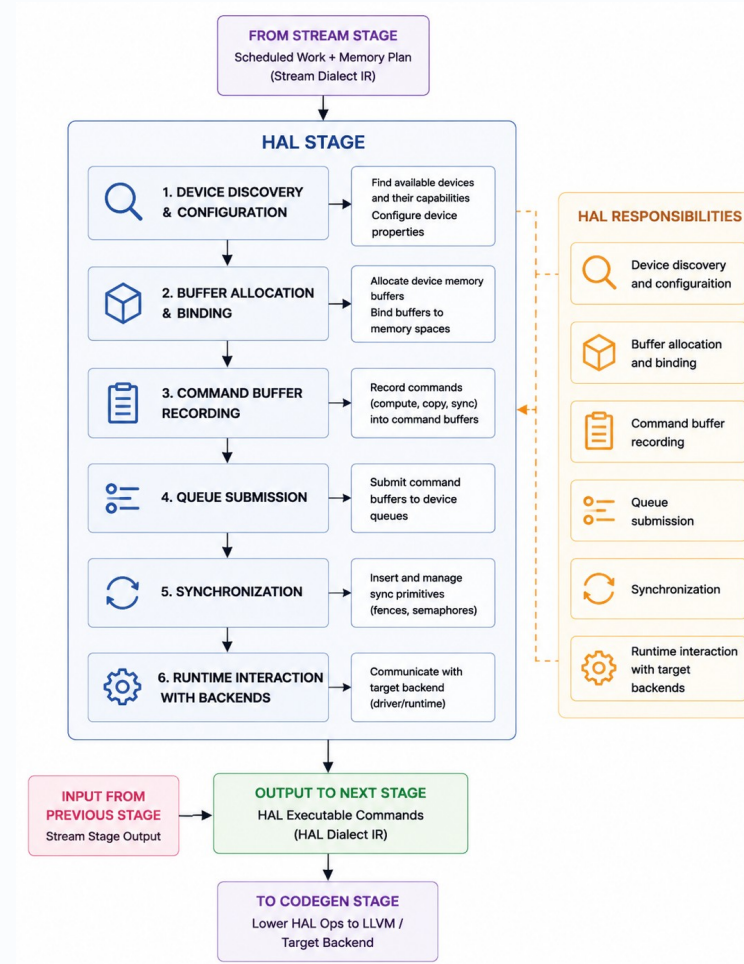
HAL responsibilities

Select device targets and executable variants

Describe buffers, bindings and command buffers

Represent uploads, dispatches and synchronization

Connect compiled executables to runtime drivers



Codegen Specializes Each Dispatch for the Target

Per-dispatch code generation now knows the hardware hierarchy, data layout and instruction capabilities.

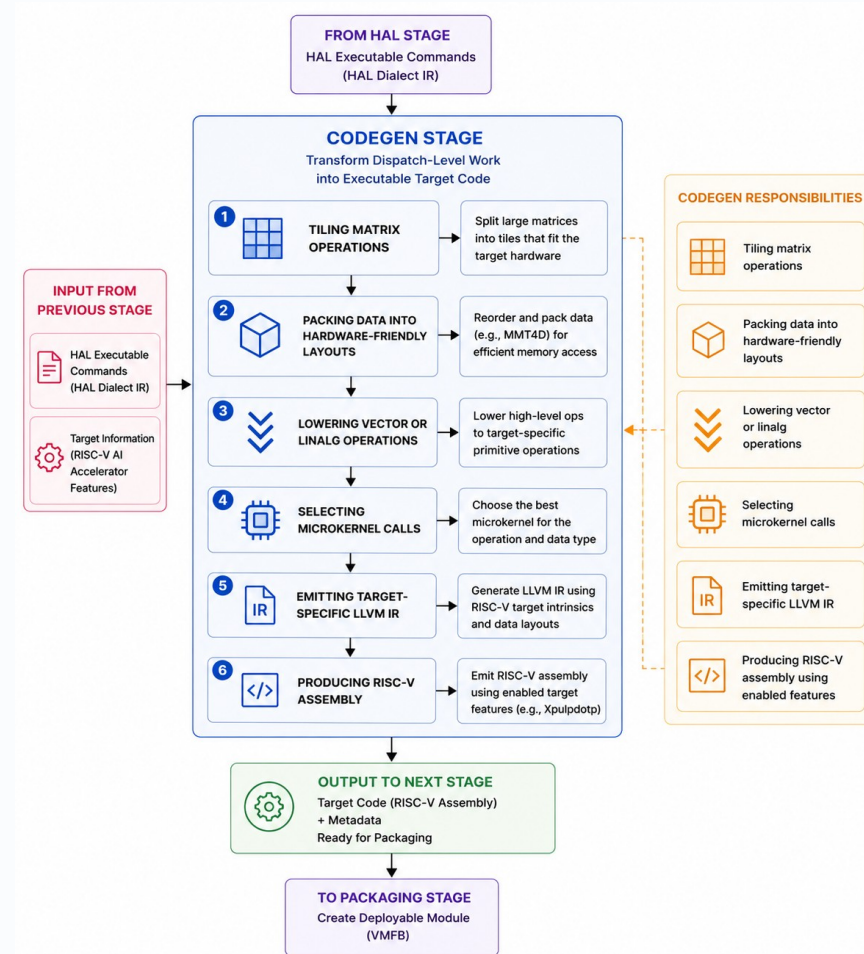
Target specialization

Choose tiles and distribution strategy

Lower structured ops to loops and vectors

Map memory spaces and vector widths

Emit LLVM IR, SPIR-V, PTX or another target form



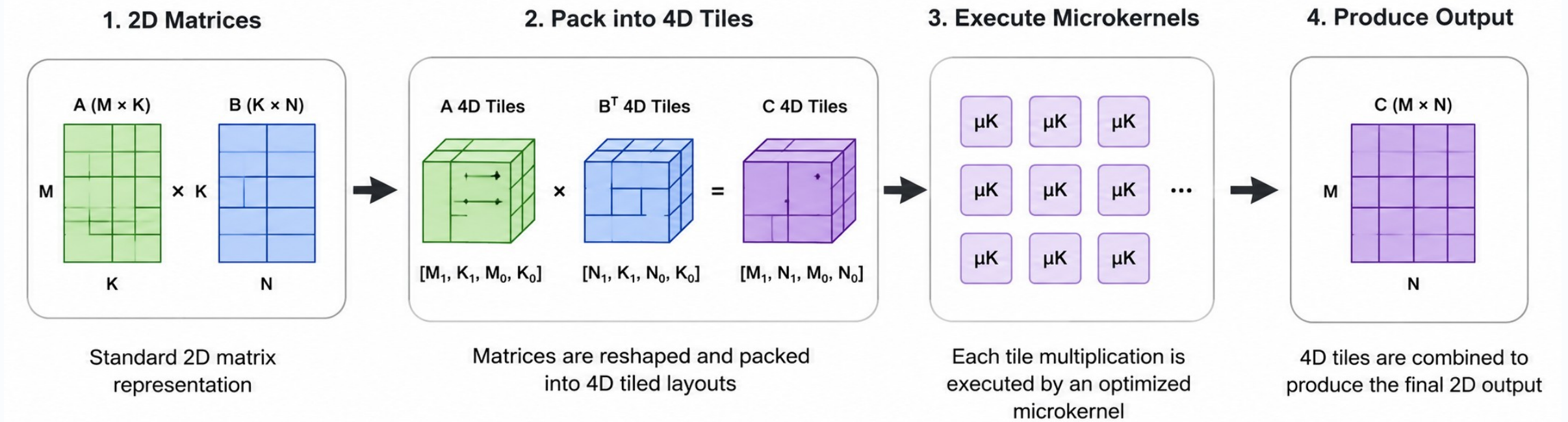
Each Optimization Belongs at a Deliberate Level

Apply a transformation where the required semantic information is still available and the target constraints are already known.

Moving a decision too early can destroy portability; moving it too late can lose high-level optimization opportunity.

LEVEL	SEES	OPTIMIZES	PRODUCES
Import	Framework graph	Legalization, canonicalization	Imported MLIR
Tensor / Linalg	Operators + iteration	Fusion, decomposition, layout	Structured tensor IR
Flow	Tensor dataflow	Dispatch formation, fusion	Dispatch regions
Stream	Resources + dependencies	Scheduling, lifetime, allocation	Async resource program
HAL	Devices + commands	Executable selection, bindings	HAL program
Codegen	One dispatch + target	Tiling, mapping, vectorization	Target IR
LLVM / backend	Low-level ops	Instruction selection, registers	Object / device binary
Runtime	Module + device	Loading, dispatch, synchronization	Executed result

Tiling Converts Large Operators into Reusable Work Units



Fusion Removes Intermediate Transfers When Locality Wins

UNFUSED

FUSED

TRADE-OFF

Write an intermediate

Producer stores a tensor; consumer reloads it in a later dispatch.

Keep values close

Compatible operations share a dispatch or tile and avoid unnecessary global-memory traffic.

Watch resource pressure

Excess fusion can increase registers, shared memory, code size or scheduling constraints.

Vectorization Maps Structured Work to SIMD or SIMT

ONE SEMANTIC OP · DIFFERENT EXECUTION HIERARCHIES

Vector IR preserves data-parallel structure while target codegen selects the concrete mapping.



CPU / RISC-V

Map vectors to SIMD or RVV registers, vector loads/stores, reductions and target intrinsics.



GPU

Distribute tiles across workgroups, subgroups and threads, then use device-specific vector forms.



Accelerator

Rewrite matched operations to a native tile, microkernel call or custom target dialect operation.

Scheduling Assigns Work to the Hardware Hierarchy

PROGRAM

DEVICE

CORE / LANE

Form dispatches

Separate host orchestration from dense compute and preserve dependencies.

Distribute tiles

Assign dispatch work to CPU tasks, GPU workgroups or accelerator queues.

Order instructions

Map inner work to loops, threads, vector lanes, pipelines and functional units.

Memory Planning Is Part of Code Generation

SIZE · LIFETIME · LOCATION · MOVEMENT

A compute-efficient kernel can still lose if buffers are allocated, laid out or transferred poorly.

Allocation

1

Choose buffer sizes, alignment, memory type and reuse opportunities.

Lifetime

2

Release transient storage safely after the last dependent command.

Layout

3

Pack and encode tensors for vector units, GPUs or accelerator-native tiles.

Transfer

4

Schedule upload, download, staging and synchronization without unnecessary copies.

Part 3 · Follow One MatMul to a RISC-V Instruction

Use MMT4D, vector.contract and VMADOT to connect high-level semantics with target code.



A MatMul Becomes a Tiled Kernel Before Execution

The compiler progressively converts a large matrix operation into packed tiles, repeated kernel calls and target instructions.

Transformation chain

Start with high-level MatMul semantics

Select tile sizes for the target

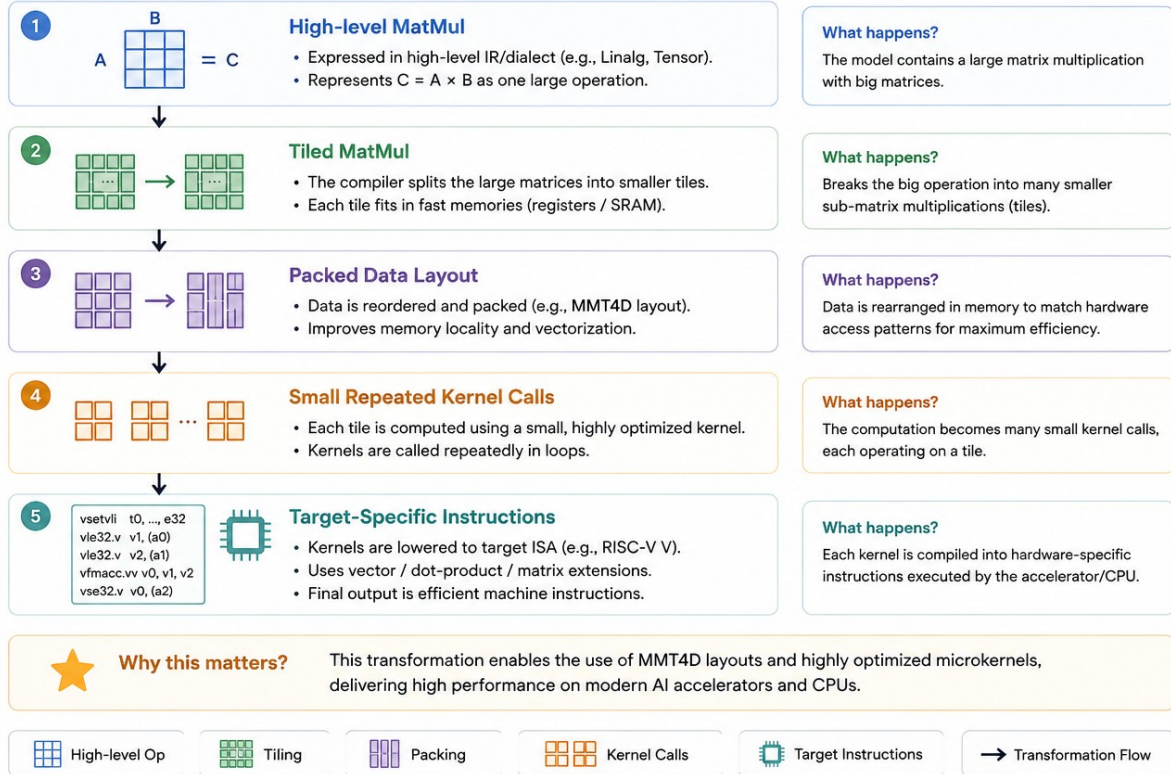
Pack operands into an MMT4D-friendly layout

Execute microkernels and combine output tiles

From MatMul to Kernel Execution



A high-level matrix multiplication is not normally executed as one giant operation. The compiler transforms it into a series of optimized steps for efficient hardware execution.



MMT4D Makes the Tile Contract Explicit

MMT4D represents matrix multiplication over packed four-dimensional tiles rather than flat two-dimensional matrices.

Why it helps

Separates outer tile traversal from inner tile compute

Makes microkernel dimensions explicit

Improves reuse and predictable memory access

Lets tile shape follow vector-register capacity

Conceptual MMT4D Layout

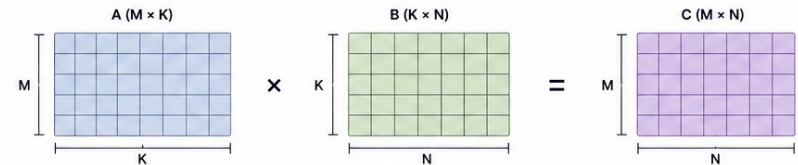


- A conventional matrix is viewed as rows and columns.
- MMT4D groups the matrix into 4D tiles to match hardware memory hierarchy and enable efficient microkernel execution.

1. CONVENTIONAL VIEW (ROW × COLUMN)

Original Matrix

Size: $M \times K$
multiplied with
 $K \times N$



Conceptual View

The matrix is seen as a 2D grid of rows (M) and columns (N or K).

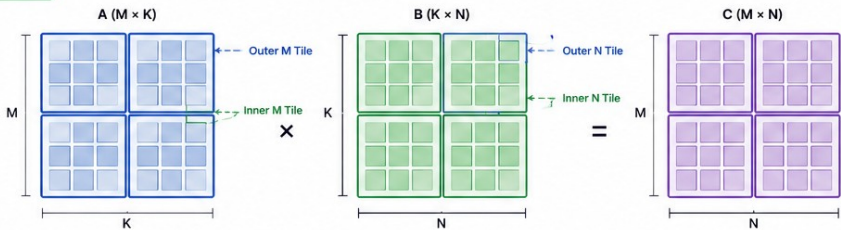
2. TILED VIEW WITH MMT4D

4D Tiling

MMT4D groups the matrix into 4D tiles.

Tiles are organized as:

Outer M × Outer N
× Inner M × Inner N



Tiled View

The matrix is divided hierarchically into 4 levels (MMT4D):

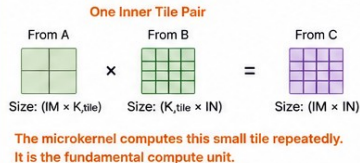
- Outer M Tile
- Outer N Tile
- Inner M Tile
- Inner N Tile

The inner tile is the compute unit for the microkernel.

3. TILE HIERARCHY EXPLAINED

- Outer M Tile (rows) Top-level row partitioning of the matrix.
- Outer N Tile (columns) Top-level column partitioning of the matrix.
- Inner M Tile (rows) Sub-division within an Outer M tile.
- Inner N Tile (columns) Sub-division within an Outer N tile.

4. MICROKERNEL COMPUTATION (INNER TILE)



5. KEY TAKEAWAYS

- Conventional view is 2D (rows × columns).
- MMT4D introduces a 4D tiling hierarchy (Outer M , Outer N , Inner M , Inner N).
- The inner tile is computed repeatedly by the microkernel for high performance.
- This layout matches hardware memory hierarchy and improves data locality.



Why MMT4D?

It enables efficient packing, reuse of data in fast memories, and high-performance microkernel execution on modern AI accelerators and CPUs.

vector.contract Exposes Parallel and Reduction Structure

The operation describes a contraction generically enough for reuse, yet precisely enough for target matching.

Representative contraction

```
%out = vector.contract {  
  iterator_types = ["parallel", "parallel", "reduction"],  
  indexing_maps = [A(i,k), B(k,j), C(i,j)]  
} %lhs, %rhs, %acc  
: vector<4x8xi8>, vector<8x4xi8> into vector<4x4xi32>
```

Iterators



i and j are independent output dimensions; k is accumulated.

Indexing maps



Operand maps prove the operation has matrix-multiply semantics.

Accumulator



The i32 initial value and result must be preserved through replacement.

Dialect Conversion Selects Only Legal Target Operations

MATCH · REWRITE · LEGALIZE · VERIFY

A target-specific replacement is allowed only when the operation satisfies the hardware contract.

Conversion target



Declares which dialects and operations are legal after the pass.

Rewrite pattern



Matches a supported contraction and creates the target operation.

Type converter



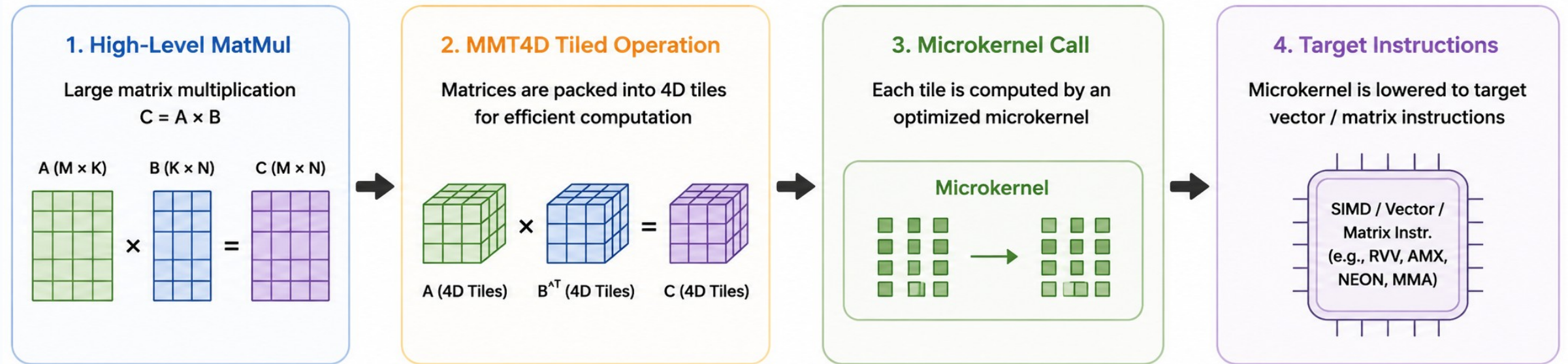
Adapts types when source and target representations differ.

Failure path



Leaves unsupported cases for another lowering or emits a useful diagnostic.

VMADOT Replaces a Matched Integer Contraction



The Match Must Prove Types, Shapes and Semantics

Instruction selection is a proof obligation, not a text substitution.

Minimum checks

Supported input types and signedness

Required accumulator and result type

Expected parallel / reduction iterator pattern

Matrix-multiply indexing maps

Ranks and tile dimensions supported by the instruction

Fail safely



Reject the pattern when one hardware precondition is not satisfied.

Adapt representation



Use shape_cast or explicit extensions only when semantics remain unchanged.

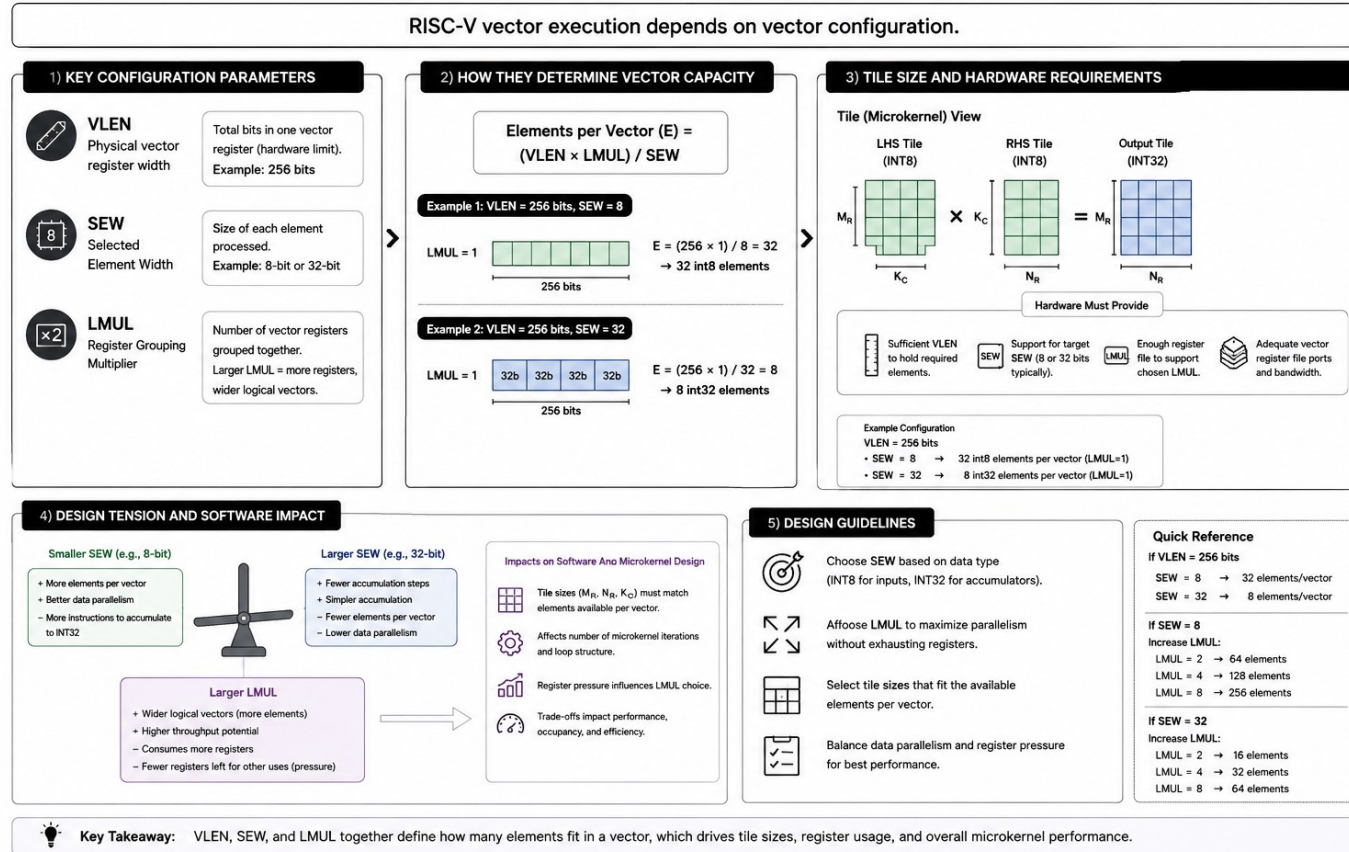
Preserve accumulation



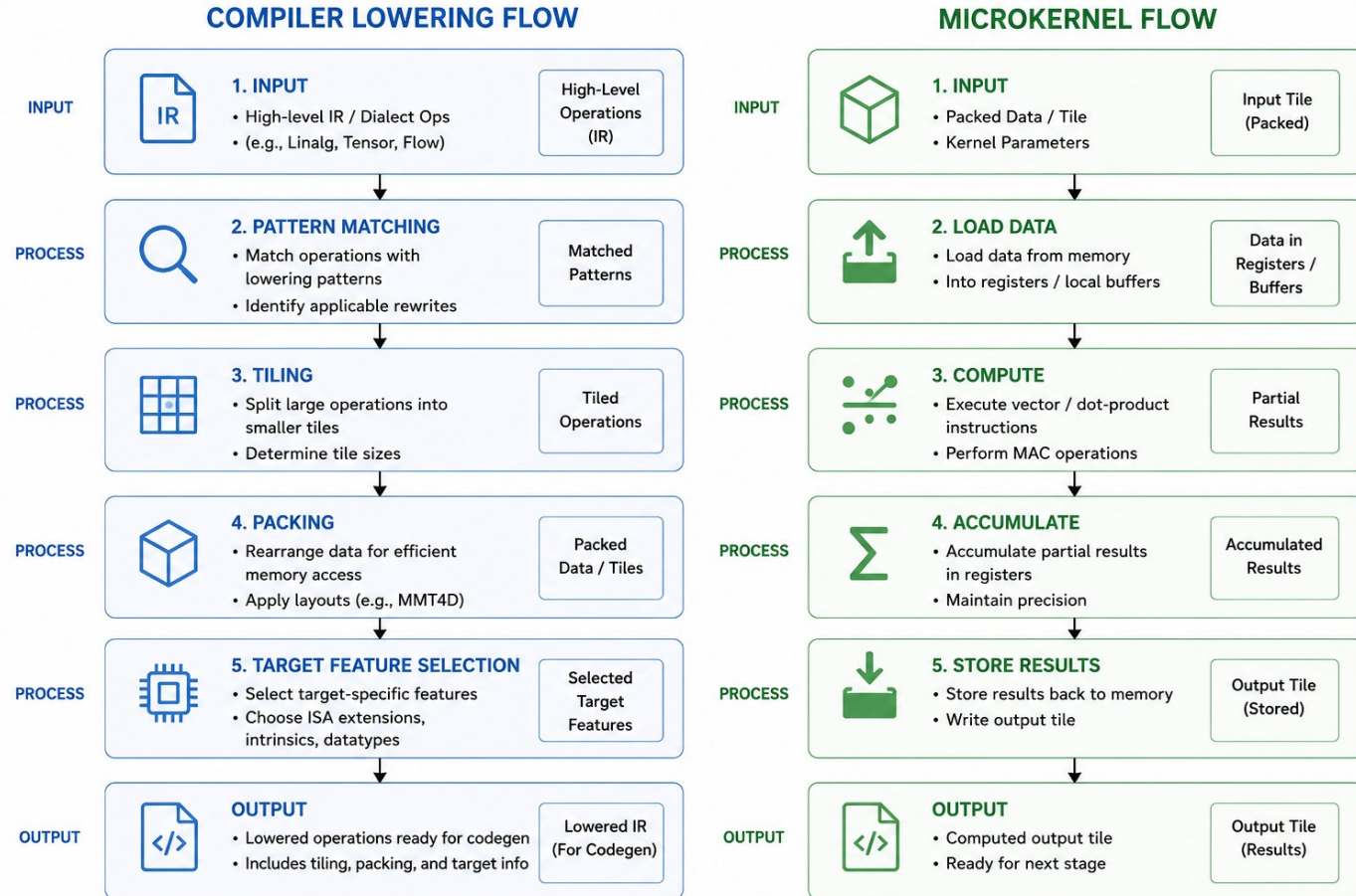
Do not drop, duplicate or reinterpret the initial accumulator.

Tile Size Is Constrained by Vector-Register Capacity

VLEN, SEW, and LMUL: Tiles, Hardware Requirements, and Software Impact

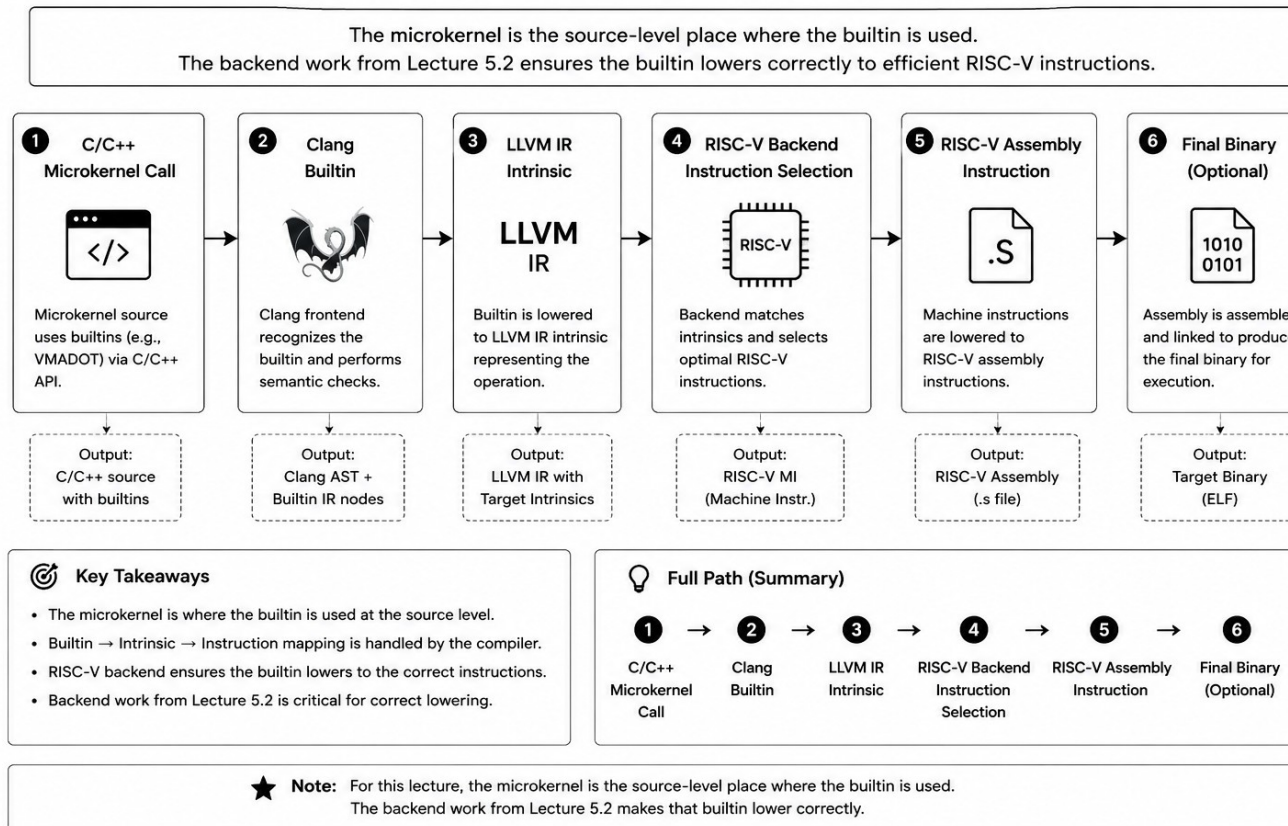


Compiler Lowering and the Microkernel Divide the Work



Builtin → Intrinsic → Backend → Assembly

Builtin to Instruction Path Revisited



Part 4 · Package and Execute on Real Hardware

Compile the module for a target, select a compatible HAL device, load inputs and verify the result.



Ahead-of-Time Compilation Produces a VMFB Module

VMFB packages VM-level host logic together with one or more target executable payloads and metadata.

AOT flow

Import high-level model into MLIR

Optimize and lower through IREE

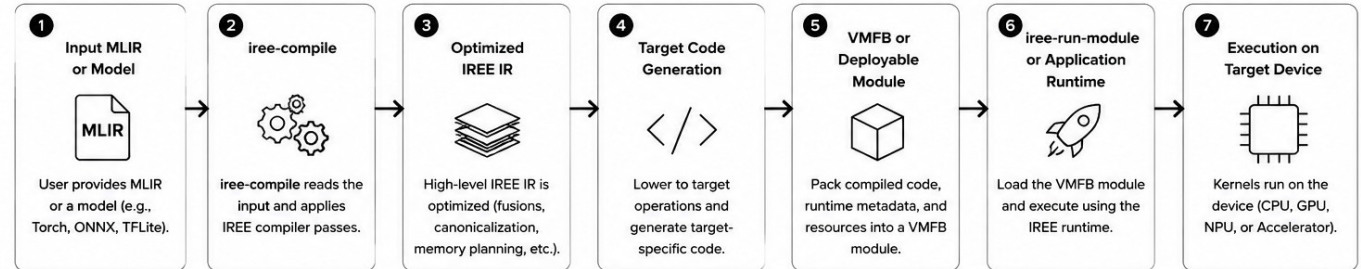
Generate target-specific executable code

Serialize a deployable .vmfb module

IREE AOT (Ahead-Of-Time) Compilation

AOT means ahead-of-time compilation.
The model is compiled into a deployable binary module before it runs.

IREE AOT Compilation Flow



What is IREE AOT?

- Ahead-of-Time Compilation:**
All heavy compilation is done before runtime.
- Faster Startup:**
No JIT compilation at runtime.
- Better for Deployment:**
Produces a portable, self-contained module (VMFB).
- Ideal for Embedded and Accelerators:**
Lower runtime overhead, predictable performance.

Why Use AOT?

- ✓ Smaller Runtime Footprint
- ✓ Deterministic Startup Time
- ✓ Stronger Security
- ✓ Easier Distribution
- ✓ Better Performance

Key Takeaway

AOT compilation in IREE converts your model or MLIR into a deployable module before it runs. This enables fast startup, low overhead, and efficient execution on the target hardware.

Command Example

```
iree-compile model.mlir \
--iree-hal-target-backends=llvm-cpu \
-o model.vmfb
```

Compiles the input MLIR for the target (e.g., CPU) and produces a VMFB module.

Run the Module

```
iree-run-module model.vmfb
```

Executes the compiled module with IREE runtime.

★ **Note:** AOT compilation is especially useful for embedded and accelerator deployment because most of the compilation work is done before runtime, reducing startup time and runtime overhead.

The Runtime Loads, Dispatches and Synchronizes Work

Execution follows a small repeatable sequence even when the underlying device API changes.

Runtime sequence

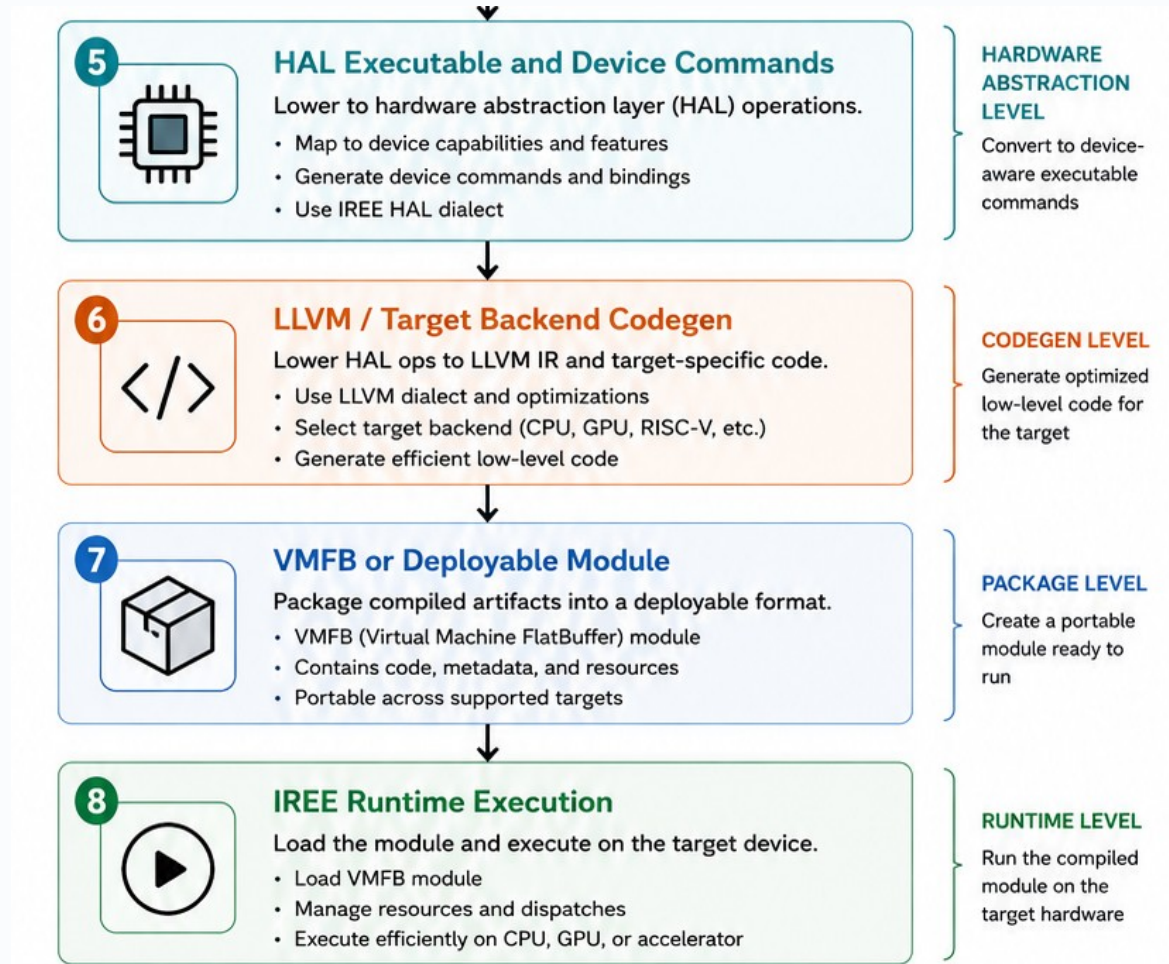
Create a compatible HAL device

Load VMFB and parameter archives

Prepare or import input buffers

Invoke the exported function

Wait, read outputs and release resources



Target Backends and HAL Devices Must Agree

Compilation produces code for a target backend; execution requires a runtime driver that can load and run that code.

A compiler flag can produce a valid module that still cannot execute when the runtime lacks the matching driver or loader.

TARGET	COMPILE	RUN DEVICE	KEY CONTRACT
x86 / ARM CPU	llvm-cpu	local-task / local-sync	Triple, CPU and features
RISC-V CPU	llvm-cpu	local-task / local-sync	ABI, +v and VLEN
NVIDIA GPU	cuda	cuda	sm architecture
Cross-vendor GPU	vulkan-spirv	vulkan	SPIR-V + device limits
AMD GPU	rocm	hip	GPU architecture
Apple GPU	metal-spirv	metal	Metal target/runtime
Portable reference	vmvx	local-task / local-sync	VMVX support
Custom accelerator	extension / external code	custom HAL/device	Commands, buffers, sync

CPU and GPU Builds Differ at the Target Boundary

The imported MLIR is common; target flags and the selected runtime device change the generated executable and launch path.

CPU · llvm-cpu + local-task

```
iree-compile model.mlir -o model_cpu.vmfb \  
  --iree-hal-target-device=local \  
  --iree-hal-local-target-device-backends=llvm-cpu \  
  --iree-llvmcpu-target-cpu=host --iree-opt-level=O2  
iree-run-module --device=local-task \  
  --module=model_cpu.vmfb --function=main --input=@input.npy
```

NVIDIA GPU · CUDA

```
iree-compile model.mlir -o model_cuda.vmfb \  
  --iree-hal-target-device=cuda \  
  --iree-cuda-target=sm_89  
iree-run-module --device=cuda \  
  --module=model_cuda.vmfb --function=main \  
  --input=@input.npy
```

RISC-V and Custom Accelerators Expose the Co-Design Knobs

These targets make the hardware–software contract especially visible: ISA features, memory layout, commands and synchronization must all align.

RISC-V + RVV

`--iree-llvmcpu-target-triple=riscv64`

`--iree-llvmcpu-target-abi=lp64d`

`--iree-llvmcpu-target-cpu-features=`

`+m,+a,+f,+d,+c,+zvl512b,+v`

`--iree-opt-data-tiling`

Run with a RISC-V runtime; match QEMU/device VLEN.

Custom accelerator

Choose the compiler entry point: input dialect, rewrite, target code or external kernel

Define buffer layout, address spaces and data movement

Implement executable packaging and a HAL/device path

Specify commands, queues, fences, errors and profiling

Test CPU reference → compiler evidence → device result

IREE Turns Model Intent into a Hardware Contract

Preserve semantics high in MLIR.

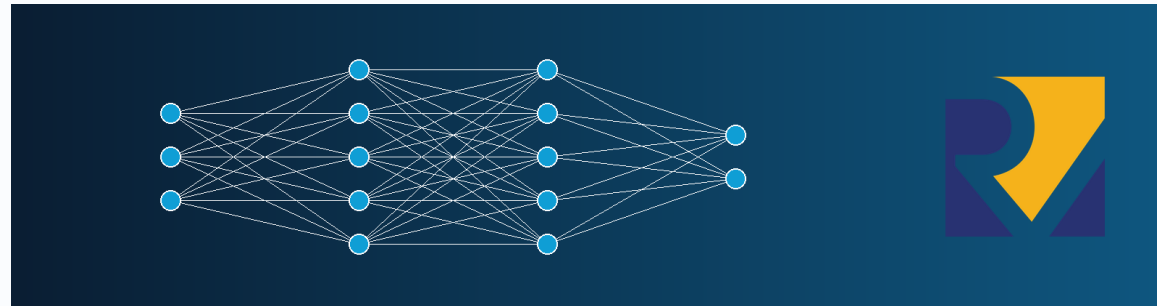
Specialize scheduling, memory and code for the target.

Validate the same model across CPU, GPU, RISC-V and accelerators.

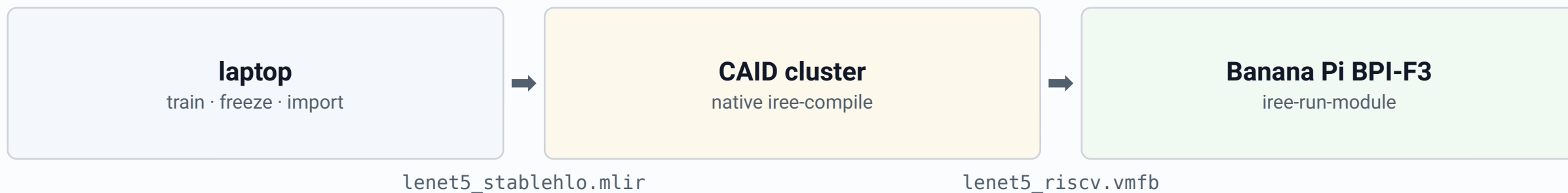


Hands-On: LeNet-5 to a RISC-V Board

Train in Python on a laptop, import to MLIR, compile on the cluster, run on a Banana Pi BPI-F3, and prove the answer survived the journey.



Three Machines, One Artifact Chain



The same 4096 bytes travel with it

`test_digit.bin` is one 32x32 float32 MNIST image. Byte for byte, it is the input on all three machines — which is the only reason the comparison at the end means anything.

Nothing but files crosses a machine boundary.

Train and Freeze, on the Laptop

what you run [\[BASH\]](#)

```
python lenet.py
```

What it produces

./lenet5/ — a TensorFlow SavedModel
test_digit.bin — one MNIST image
the reference logits, printed

Why it re-loads the model

The script freezes variables into constants, then loads the SavedModel back and checks it still agrees with the eager model. That is the first of five correctness checks, not ceremony.

Stop here

Do not continue until the Python model predicts 7. Everything downstream is measured against this run.

The demo starts with an answer you already trust.

Write the Answer Down Before You Compile Anything

laptop · reference execution

True label: 7

Predicted: 7

Logits: [-1.5658627, -3.0228333, 1.3669847, 2.0232244, -6.5975933,
-2.265958, -14.284359, 10.659572, -0.6285641, 2.3045204]

Why this is a good test

Index 7 scores 10.66. The runner-up scores 2.30. It is not a close call, so a broken pipeline will not pass it by accident.

The input is a file too

test_digit.bin is exactly 4096 bytes: 1 x 32 x 32 x 1 float32 values. Ten numbers and 4 KB are the whole contract.

A faster wrong result is not an optimisation. It is a failed contract.

Import to MLIR, Still on the Laptop

IREE tools, installed with pip [\[BASH\]](#)

```
# TensorFlow SavedModel -> MLIR bytecode
iree-import-tf ./lenet5 --tf-import-type=savedmodel_v2 \
               --tf-savedmodel-exported-names=serve \
               -o lenet5_stablehlo.mlir

# bytecode -> human-readable text, for reading and diffing
iree-ont --emit-bytecode=false lenet5_stablehlo.mlir \
```

Why two files

Bytecode is what tools pass around: compact, versioned, fast to parse.
Text is what you read. Identical module, two encodings.

Inspect before continuing

Is @serve there with the shapes you expect? Any operation you do not recognise? Are the weights inline or referenced?

This is the file that stops being TensorFlow's problem.

Read What the Importer Actually Produced

lenet5_stablehlo_hr.mlir — abridged [\[MLIR\]](#)

```
func.func @serve(%arg0: tensor<1x32x32x1xf32>) -> tensor<1x10xf32> {
  %cst_13 = stablehlo.constant dense<...> : tensor<5x5x1x6xf32>

  %0 = stablehlo.convolution(%arg0, %cst_13)
    dim_numbers = [b, 0, 1, f]x[0, 1, i, o]->[b, 0, 1, f],
    window = {stride = [1, 1], pad = [[2, 2], [2, 2]]}
    : (tensor<1x32x32x1xf32>, tensor<5x5x1x6xf32>)
      -> tensor<1x32x32x6xf32>

  %3 = stablehlo.maximum %2, %cst_7 : tensor<1x32x32x6xf32>    // relu
  return %19 : tensor<1x10xf32>
}
```

The read-out-loud moment

Find that first `stablehlo.convolution` and match its `5x5x1x6` filter against `Conv2D(6, 5)` in the Python. Same layer, two vocabularies.

52 Lines. 667 Kilobytes.

laptop

```
$ wc -l lenet5_stablehlo_hr.mlir      -> 52
$ du -b lenet5_stablehlo_hr.mlir     -> 667426
```

ASK THE ROOM

Fifty-two lines of text weighing two-thirds of a megabyte. Where is it all?

the answer

```
$ grep -o 'stablehlo\[a-z_\]*' *_hr.mlir | sort | uniq -c | sort -rn
  17 stablehlo.constant      <-- every weight tensor, inline, in decimal
   7 stablehlo.add           4 stablehlo.maximum
   3 stablehlo.dot           2 stablehlo.convolution
```

Compile for RISC-V, on the Cluster

transfer, then build with the native iree-compile [\[BASH\]](#)

```
scp lenet5_stablehlo.mlir <user>@<caid-cluster>:~/work/    # PLACEHOLDER

iree-compile lenet5_stablehlo.mlir -o lenet5_riscv.vmfb \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-llvmcpu-target-triple=riscv64 \
  --iree-llvmcpu-target-abi=lp64d \
  --iree-llvmcpu-target-cpu-features=+m,+a,+f,+d,+c,+v    # match the board
```

Why the cluster

A source build of IREE is heavy. It is also where you can afford to build several target variants and compare them.

The features flag is a contract

Promise +zvl512b to a 256-bit board and codegen sizes its tiles for registers that do not exist.

Open the Container in Front of the Room

on the cluster [\[BASH\]](#)

```
unzip -l lenet5_riscv.vmb
```

VM bytecode

Portable host logic: sequences buffers, dispatches and waits.

Executable

Real riscv64 machine code, one payload per target variant.

Parameters

The weights — the part that dominates the file size.

Metadata

Entry-point signatures, ABI and device requirements.

Why bother

Every box in the IREE architecture diagram turns out to be a file you can list. Point at each entry and name the compiler stage that produced it.

Run It on the Board

on the Banana Pi BPI-F3 [\[BASH\]](#)

```
iree-run-module \  
  --device=local-task \  
  --module=lenet5_riscv.vmfb \  
  --function=serve \  
  --input=1x32x32x1xf32=@test_digit.bin
```

What the room should notice

There is no Python on this machine. No TensorFlow. No model file in any framework format. There is a small runtime and a container, and the answer comes out anyway.

If it fails here

Rebuild targeting vmvx and run again. Correct on vmvx and wrong on llvm-cpu isolates the bug to RISC-V code generation, not to your model.

Argmax should be 7. The contract held across three machines and two ISAs.

Compare, and Explain the Difference

comparing laptop and board

reference : 10.659572

device : 10.659574

`np.allclose(ref, dev, atol=1e-4)` -> True

`np.array_equal(ref, dev)` -> False

ASK THE ROOM

Same weights. Same input. Same arithmetic. Why is the last digit different — and which one is correct?

Neither. Both are valid roundings of the same exact sum.